

AI FLUENCY PROMPT LADDER

DOCUMENT: MACHINE LEARNING TRACK

Task: Model Selection & Preprocessing for Tabular Regression

PART 1: FINAL REUSABLE PROMPT (STAGE 4)

Role & System Persona

Act as an expert Machine Learning Tutor. Your goal is to guide a student through an end-to-end model selection, preprocessing, and feature engineering workflow for a tabular regression problem.

Dataset & Context

- Problem Type: Supervised Regression
- Target Variable: Continuous numerical target (e.g., MonthlyIncome / Salary)
- Feature Types: Mixed dataset containing numerical, nominal categorical, and ordinal variables (~30+ columns total, e.g., IBM HR Dataset schema).
- Sample Size: Small-to-medium dataset (~1,500 rows).
- Known Data Anomalies:
 - Missing Values: Minor (<2% across select columns).
 - Outliers/Errors: ~20% corrupt entries (e.g., negative values for non-negative continuous targets).

Core Task Requirements

1. Preprocessing Strategy: Outline concrete steps to handle missing values, encode categorical variables, scale features where necessary, and programmatically handle corrupted negative target values.
2. Feature Engineering: Recommend at least 3 domain-relevant engineered interaction features.

3. Family Trade-off Analysis: Compare three distinct model families for this task:

- Linear Models (e.g., Ridge/Lasso Regression)
- Tree-Based Ensembles (e.g., Random Forest vs. Gradient Boosting/XGBoost)
- Deep Learning (e.g., Multi-Layer Perceptron)

4. Model Selection Evaluation: Recommend appropriate regression metrics (e.g., R^2 , RMSE, MAE) rather than classification accuracy.

Review & Tutor Instructions

- Explain the underlying statistical and mathematical assumptions each model family makes about the data.
- Conclude your response with 3 specific check-for-understanding questions to test the user's grasp of how data size, distribution, and non-linearity dictate model choice.

PART 2: PROMPT LADDER & EVOLUTION (STAGES 1–3)

STAGE 1: BASELINE (VERSION 0)

Prompt:

"what is the best model to use for this data?"

Age Attrition BusinessTravel DailyRate Department DistanceFromHome Education
EducationField EmployeeCount EmployeeNumber EnvironmentSatisfaction Gender
HourlyRate JobInvolvement JobLevel JobRole JobSatisfaction MaritalStatus
MonthlyIncome MonthlyRate NumCompaniesWorked Over18 OverTime
PercentSalaryHike PerformanceRating RelationshipSatisfaction StandardHours
StockOptionLevel TotalWorkingYears TrainingTimesLastYear WorkLifeBalance
YearsAtCompany YearsInCurrentRole YearsSinceLastPromotion YearsWithCurrManager

41 Yes Travel_Rarely 1102 Sales 1 2 Life Sciences 1 1 2 Female 94 3 2 Sales Executive 4
Single 5993 19479 8 Y Yes 11 3 1 80 0 8 0 1 6 4 0 5

49 No Travel_Frequently 279 Research & Development 8 1 Life Sciences 1 2 3 Male 61 2
2 Research Scientist 2 Married 5130 24907 1 Y No 23 4 4 80 1 10 3 3 10 7 1 7

37 Yes Travel_Rarely 1373 Research & Development 2 2 Other 1 4 4 Male 92 2 1
Laboratory Technician 3 Single 2090 2396 6 Y Yes 15 3 2 80 0 7 3 3 0 0 0 0

33 No Travel_Frequently 1392 Research & Development 3 4 Life Sciences 1 5 4 Female
56 3 1 Research Scientist 3 Married 2909 23159 1 Y Yes 11 3 3 80 0 8 3 3 8 7 3 0

27 No Travel_Rarely 591 Research & Development 2 1 Medical 1 7 1 Male 40 3 1
Laboratory Technician 2 Married 3468 16632 9 Y No 12 3 4 80 1 6 3 3 2 2 2 2"

Representative Output Excerpt:

"The 'best' model depends on your target variable. From the sample you shared, it looks like you're trying to predict Attrition (Yes/No), which makes this a binary classification problem. For a dataset like this (the IBM HR Employee Attrition dataset), here's how the models generally compare: Random Forest (a great first choice), XGBoost/LightGBM (best performance), Logistic Regression (baseline). ... For this specific HR attrition dataset, XGBoost (or LightGBM) is often the top performer, with Random Forest being an excellent alternative if you want a simpler model."

STAGE 2 & 3: FIVE-STEP EVOLUTION WITH 4 NOTES PER RUN

VERSION 1: Clearer Goal

Prompt:

"what is the model that will give the highest accuracy for this data? <sample rows pasted>"

Representative Output Excerpt:

"There is no single model that always gives the highest accuracy. The best-performing model depends on the dataset, preprocessing, class imbalance, and hyperparameter tuning. ... The IBM attrition dataset is imbalanced (roughly 84% 'No', 16% 'Yes'). A model can achieve around 84% accuracy by predicting 'No' for everyone, yet be practically useless. For this problem, you should evaluate ROC-AUC, F1-score, Precision, Recall, and Confusion matrix — not accuracy alone."

Note 1: *What changed in prompt:* Asked specifically for the model that yields the highest accuracy.

Note 2: *What actually improved in output:* The output pivoted immediately to high-performance gradient boosting ensembles (CatBoost, XGBoost, LightGBM) as top recommendations. It also explicitly warned that accuracy alone is misleading for imbalanced binary targets and introduced ROC-AUC/F1-scores.

Note 3: *What still failed:* The guidance was still purely technical and lacked structure on how to present or evaluate these models for a formal class assignment.

Note 4: *What to try next:* Define the audience to get advice tailored to academic assignment criteria.

VERSION 2: Defined Audience

Prompt:

"what is the model that will give the highest accuracy for this data? The model is to be used for my machine learning assignment. <sample rows pasted>"

Representative Output Excerpt:

"For a machine learning assignment, a good approach is to compare several models instead of reporting only one. For example: Logistic Regression (baseline), Decision Tree, Random Forest, XGBoost (best model). This demonstrates that you evaluated multiple algorithms before selecting the best one." [Model ranking table: XGBoost 88–92%, LightGBM 88–91%, CatBoost 88–91% — unchanged from Version 1.]

Note 1: *What changed in prompt:* Added that the output is meant to guide a machine learning assignment submission.

Note 2: *What actually improved in output:* The output stopped trying to give a single "magic" model and instead gave an assignment strategy — comparing multiple algorithms (Logistic Regression → Decision Tree → Random Forest → XGBoost) to demonstrate proper experimental methodology to a grader.

Note 3: *What still failed / Honest Non-Improvement:* **This addition did not actually improve the core technical output or model recommendations.** Adding "for my machine learning assignment" only caused the model to restate the exact same algorithms (XGBoost/LightGBM) with a small academic meta-commentary about evaluating multiple models. The output remained stuck because the prompt still hadn't provided the actual dataset context, proving that defining an audience without providing underlying data facts adds fluff without improving technical quality.

Note 4: *What to try next:* Provide real dataset context, including the actual target variable, preprocessing goals, and specific data anomalies.

VERSION 3: Providing Real Context

Prompt:

"what is the model that will give the highest accuracy for this data? The model is to be used for my machine learning assignment. The target is MonthlyIncome. The task is to do feature engineering and compare 3 models and pick the model with the highest

accuracy. The aim of the assignment is to implement preprocessing and feature engineering to enhance the model performance. Initial EDA shows <2% missing data in 2 columns and 20% rows with negative salaries. <sample rows pasted>"

Representative Output Excerpt:

"Your target is MonthlyIncome, which is continuous, so this is a regression problem, not classification. Therefore, you should evaluate using regression metrics such as R^2 , RMSE, and MAE. ... You mentioned 20% negative salaries. MonthlyIncome cannot legitimately be negative. Possible approaches: convert to absolute values only if you can justify the minus sign is erroneous, or treat them as invalid values and remove/impute those rows. Document your reasoning. Simply leaving negative salaries will seriously degrade model performance. ... Useful engineered features include $\text{YearsPerCompany} = \text{TotalWorkingYears} / \text{NumCompaniesWorked}$ and $\text{JobLevel} \times \text{TotalWorkingYears}$ as an income growth proxy."

Note 1: *What changed in prompt:* Clarified that the actual target is MonthlyIncome (not Attrition), required comparing 3 models after feature engineering, and provided specific EDA anomalies (<2% missing, 20% negative salaries).

Note 2: *What actually improved in output:* Major breakthrough. The AI immediately caught that this is a Regression problem (not Classification) and corrected all evaluation metrics to R^2 , RMSE, and MAE. It gave domain-specific feature engineering formulas and concrete strategies for dealing with the negative salary anomaly.

Note 3: *What still failed:* The response dumped a wall of feature ideas and general suggestions without deeply analyzing the underlying trade-offs across distinct algorithmic families.

Note 4: *What to try next:* Impose structural constraints requiring a comparative analysis across model families (Linear vs. Trees vs. Neural Networks).

VERSION 4: Specified Output Format & Constraints

Prompt:

"what is the model that will give the highest accuracy for this data? The model is to be used for my machine learning assignment. The target is MonthlyIncome. The task is to do feature engineering and compare 3 models and pick the model with the highest accuracy. The aim of the assignment is to implement preprocessing and feature engineering to enhance the model performance. Initial EDA shows <2% missing data in 2 columns and 20% rows with negative salaries. Focus your explanation on comparing different families of models (e.g., Linear Models, Tree-Based Ensembles, Neural

Networks) and provide a breakdown of the trade-offs of using each model for this specific problem. <sample rows pasted>"

Representative Output Excerpt:

"Model 1 — Multiple Linear Regression: assumes linear relationships; expected lowest R^2 , highest RMSE. Model 2 — Random Forest Regressor: handles nonlinear relationships and categorical interactions without manual scaling; expected much higher R^2 , lower RMSE. Model 3 — Gradient Boosting: typically achieves the highest predictive performance on structured/tabular datasets; expected highest R^2 , lowest RMSE. Neural Network (MLP Regressor): usually requires much larger datasets to outperform tree-based methods; for ~1,470 rows this is likely overkill."

Note 1: *What changed in prompt:* Explicitly constrained the output to compare three specific model families (Linear, Tree Ensembles, Neural Networks) and detail their trade-offs for continuous salary prediction.

Note 2: *What actually improved in output:* The output structured the comparison into clear analytical sections, explaining why Neural Networks are overkill for 1,470 rows, why Linear Regression fails on non-linear salary jumps, and why Tree Ensembles handle interaction terms without manual scaling.

Note 3: *What still failed:* The output acted strictly as a passive reference document rather than checking if I actually understood the underlying ML concepts.

Note 4: *What to try next:* Add quality criteria and review instructions directing the AI to act as an interactive tutor and test my knowledge.

VERSION 5: Quality Criteria & Review Instructions

Prompt:

"what is the model that will give the highest accuracy for this data? The model is to be used for my machine learning assignment. The target is MonthlyIncome. The task is to do feature engineering and compare 3 models and pick the model with the highest accuracy. The aim of the assignment is to implement preprocessing and feature engineering to enhance the model performance. Initial EDA shows <2% missing data in 2 columns and 20% rows with negative salaries. Focus your explanation on comparing different families of models (e.g., Linear Models, Tree-Based Ensembles, Neural Networks) and provide a breakdown of the trade-offs of using each model for this specific problem. Do not just give me the answer without explanation. Act as an expert Machine Learning tutor. Guide me through the logic, explain the assumptions each model makes about the data, and end your response with 3 check-for-understanding

questions to test whether I understand how data properties dictate model selection.
<sample rows pasted>"

Representative Output Excerpt:

"Option C: Treat [negative salaries] as missing. A more statistically defensible approach: MonthlyIncome < 0 → NaN, then impute based on related variables. For example, JobLevel=4/YearsExperience=15/Department=Sales should not receive the same income estimate as JobLevel=1/YearsExperience=1. This allows feature engineering to help recover the missing information.

Check your understanding:

1. Why would a tree-based model usually capture the relationship between JobLevel and MonthlyIncome better than a linear regression model?
2. If your dataset increased from 1,400 employees to 10 million employees, how might your preferred model choice change and why?
3. Why is treating negative MonthlyIncome values as missing data potentially better than simply deleting all affected rows?"

Note 1: *What changed in prompt:* Instructed the AI to adopt a pedagogical role ("ML tutor"), explain statistical assumptions (e.g., linearity, variance), and append 3 check-for-understanding questions.

Note 2: *What actually improved in output:* The response transformed into a full lesson plan. It explicitly broke down ordinal vs. nominal encodings, detailed three defense options for the 20% negative salary issue, explained how Gradient Boosting sequentially minimizes residual error, and ended with 3 thought-provoking comprehension questions linking dataset size to model architecture selection.

Note 3: *What still failed:* The prompt itself contains cluttered raw CSV text pasted directly into the instruction block, which is messy for someone else to re-use.

Note 4: *What to try next:* Consolidate the lessons learned into a clean, reusable system brief template for any tabular regression assignment (see Part 1).